

Job-Apply Harness — Architecture Blueprint v2.13

DATABASE EDITION · SKILLS-FIRST, INSTALL-READY

How we build, install, and operate autonomous job-application harnesses on Muse — with **Career Harness** (the display name of the private fullstack artifact whose technical slug remains `harness-core`; SQLite + published actions) as the single store, and a versioned skill catalog as the only place logic lives.

Contents

0. The system in one sentence	14. Coordinator and agent orchestration
1. Architectural bets	15. Run lifecycle: trigger to chat in seven steps
2. Muse primitives — what each carries, and what we deliberately don't use	16. Reliability
3. Skills-first architecture: thin goals, fat skills	17. Filesystem and visibility contract
4. System overview — four layers	18. Memory and judgment — the hard boundary
5. Career Harness (<code>harness-core</code>): the single store	19. Install-ready: the kit as software
6. Published actions API	20. Why this beats the market
7. Profile as data, cron bodies compiled	21. Reconciliation with N-Pulse (what carried over, what died)
8. Campaigns and the schedule fleet	22. Build order
9. Application state machine and the intent log	23. Productizing: packaging and onboarding
10. Judges and scoring	24. Learning loop
11. Inbox: conversations and the replies ledger	25. Open decisions — what I recommend
12. Approvals and safety	26. Risks
13. Career Harness — one artifact, seven tabs, focused write APIs	27. Roadmap
	The principle underneath all of it

0. The system in one sentence

Schedules wake workers. Workers act only through `harness-core`, following skills. Dashboards read `harness-core`. The main agent decides what reaches the client's phone. Files hold the evidence; the database holds the truth; skills hold the logic.

What's new in v2.1 (and why)

v2.0 fixed v1's three painful lessons — dual-writer divergence (37 rows vs 25), rule drift across 15 hand-edited cron bodies, and the "missing report $\neq$ missing action" problem. v2.1 is the principal-engineering pass that makes it shippable software:

- 1. Skills-first.** All logic moves into a versioned skill catalog (18 skills, one contract). Goals become thin: a `GOAL.md` contract plus a compiled cron body that invokes the `run-coordinator` skill. Nothing else. When logic lives in exactly one place, it can be tested, versioned, and improved for every customer at once.
- 2. Install-ready.** The kit becomes real software: semver versioning, an idempotent installer, schema migrations, `profile.schema.yaml` validation, post-install smoke tests. "Runs on each system" is a property you engineer, not a hope you express.
- 3. Every Muse primitive evaluated.** Hooks, deterministic workflows, paired devices — surveyed honestly: what we use now, what we pilot next, what we deliberately don't.

Scope is unchanged: Apply Engine + Inbox + Control Plane. No Presence, no Content — those remain separate add-on modules, never in the core kit.

What's new in v2.2

One wiring change, folded in per the living-spec rule:

- 1. Email scans and replies run on Outlook, never Gmail.** The `email_scan` campaign and the email-replier skill now read and reply exclusively through the `outlook-mail` skill against the connected Outlook mailbox (`neerajreddyr@outlook.com`). The Gmail connector is not connected, so the compiled `email_scan` cron body names `outlook-mail` explicitly and forbids Gmail. Reply tiers, the watermark rule, and the report rule are unchanged — only the mailbox wiring moved.

What's new in v2.3

One reliability change, folded in per the living-spec rule:

- 1. LinkedIn inbox scans are bounded.** The hourly `linkedin_replies` run stalled once on an unbounded full-inbox browser pass (~35 minutes, no threads reported), so the scan now has hard bounds: unread filter first, at most 10 threads per run (newest-first since the watermark), and a 15-minute timebox — the worker stops and reports even if threads remain. The watermark advances only over threads actually triaged, never over threads cut off by the timebox. The change lives in the `linkedin-replier` skill (new "Scan scope (inbox contract)" section, skill version 1.1.0) and in the compiled `linkedin_replies` cron body. Reply tiers, the action cap, the no-PDF rule, and the report rule are unchanged.

What's new in v2.4

One control-plane change, folded in per the living-spec rule:

- 1. Profile is managed from the app, not the YAML.** The `profile.yaml` file is now first-time-setup only. A sixth dashboard tab, **Profile**, lets Neeraj view and edit every profile section in the app (identity, screening answers, caps, targeting, locations, work auth, comp, start date, reply tiers). Saving calls a new `profile_save` action that writes to BOTH the live profile store (`profile_put`, which every scheduled run reads via `profile_get`) and back to `~/workspace/profile.yaml`, so the file always mirrors the screen and stays easy to read/inspect. The YAML write goes through a new allowlisted privileged file-write contract (exactly one path, no traversal) and is validated against the same `profile.schema.yaml` rules `compile-schedules` enforces — including quoting `Yes / No` answers so YAML 1.1 never coerces them to booleans (the bug that rejected an early compile). The file is written before the DB write: if the DB write ever fails, the next `compile-schedules` run re-syncs YAML → DB and they converge. Section 7 and section 13 document the new flow.

What's new in v2.5

A repair release, folded in per the living-spec rule. Three defects from the first live morning run (2026-09-26) plus the promised smoke-test cleanup:

- 1. Smoke-test rows are gone, via a safe purge action.** A new published action, `test_data_purge`, removes only clearly-labeled smoke/test rows (strict markers: `smoke / test` in ids, `source='smoke'`, `example.com` URLs, `[SMOKE TEST]` labels). It re-verifies every candidate before deleting, refuses the entire purge if any row — or any dependent row in approvals, reviews, replies, conversations — lacks a smoke marker, never cascades silently, never touches `event_log` history, and writes one audit event enumerating every deleted id. First production run removed the `smoke-1` posting, `smoke-variant-1`, and the `smoke_test` run; `resume_pick` can no longer rank a phantom variant above the real resume.

2. **Wrongly-blocked applications can be legally re-opened.** Four morning-run postings (Akumin, Step, CVS Health, Stryker) were blocked on bogus verdicts — the applier briefs used stale summary URLs instead of ledger rows. `blocked` was terminal with no outgoing edge, so a new audited edge was added: `blocked → discovered`, allowed only with `evidence.coordinator_correction=true` plus an `event_log` id of a `coordinator_correction` event written for that specific application. All four were re-opened through this edge; nothing was resubmitted. The re-opened rows re-enter at `discovered` and are re-screened from their ledger rows by the next campaign run.
3. **Judges never invent years; briefs never leave the ledger; browsers belong to the coordinator.** Three skill hardenings (all v1.1.0): `run-coordinator` now passes `years_matrix` to `fit-judge` only from `profile_get` — never fabricated — and every applier/browser brief cites the ledger `app_id` plus the URL copied verbatim from the ledger row via `snapshot`, re-read before acting; `fit-judge` returns `hold` (not a mass-reject) when the profile has no years matrix at all, and a profile-years approval is queued for Neeraj so real numbers can land; `portal-navigator` and `run-coordinator` now state that APPLY-phase browser driving is coordinator-level only — the coordinator spawns the browser task itself (max 2 concurrent) and never delegates browser work to a worker subagent.

What's new in v2.6

An evidence-visibility repair, folded in per the living-spec rule. The first live submission (Zeta Global, 2026-09-26) surfaced three customer-visible gaps: resume links in the dashboard didn't open, no submission screenshot was retained, and submitted rows didn't record which resume file was uploaded. The pipeline recorded hashes and paths, but the dashboard couldn't serve the files.

1. **Submitted rows now carry a full evidence bundle, attached by a schema-enforced action.** A new published action, `evidence_attach(app_id, resume_path, resume_hash, variant_id, screenshot_path?, confirmation?)`, writes the evidence fields on an application row. `resume_hash` must be a 64-character lowercase hex sha256 — the schema rejects anything else, so truncated hashes are unrepresentable. It never transitions state: attaching evidence to a `submitted` row leaves it `submitted`.
2. **Evidence files open through a record-bound action, `file_open`.** The dashboard's resume/screenshot/confirmation links call `file_open({app_id, kind: resume|screenshot|confirmation})` (or `{variant_id}` for Resume Library rows), which resolves the path recorded on that specific row and serves an allowlisted blob URL. Arbitrary filesystem paths are refused — a row can only ever open the file attached to itself. The registered base resume opens the same way.
3. **Canonical evidence paths.** Screenshots live at `goals/<campaign>/hidden_files/<run_id>/screenshots/<app_id>_<step>.png`, with `<app_id>_confirmation.png` / `<app_id>_confirmation.txt` for the confirmation capture. `portal-navigator` and `run-coordinator` (both now v1.2.0) require the full bundle before the `applying → submitted` transition is written: confirmation text, screenshot path (or an explicit "not captured" marker when the capture failed), the exact uploaded resume path, and its full sha256. The coordinator verifies the files exist before marking submitted.
4. **Honest absence.** The Zeta submission's screenshot was lost mid-run and is recorded as "not captured" — the UI shows the absence as such, never a dead link. Evidence from before this change was backfilled where the files still existed (Zeta: resume path + full hash + confirmation text); what no longer exists is marked missing, not invented.

What's new in v2.7

A dashboard repair, folded in per the living-spec rule. Neeraj missed the old ts-spaces `replies-ledger` page — a single chronological activity table (Date / Channel / Person-Topic / Action / Reason-Source) spanning email and LinkedIn — and asked why it was removed, plus whether each row could show the reply rule id (R1–R8).

1. **Why the old page went away.** The old fleet was five separate artifacts, each with its own database; the replies-ledger page read a stale copy of the data and could never call `harness-core` actions from another artifact's runtime. Any cross-artifact dashboard is broken by construction (section 13), so the page was retired — not the data. The replacement lives where the data lives: the Replies tab inside `harness-core`.
2. **The Replies tab is now the unified activity ledger.** One table, newest first, email and LinkedIn rows in a single timeline: Date (America/Chicago) · Channel (LinkedIn/Email pill, derived from the thread id) · Person/Topic (the leading descriptor of the reply reason) · Action (Sent / Held / Skipped / Auto-sent pill) · **Rule — a prominent pill showing the exact `ruleId` carried on the reply row (R1–R8, or the tier name when the row carries one instead)** · Reason/Source (full reason text, expand-on-click). The tab's existing header counters (sent/held counts, outcome funnel, awaiting-me list) are unchanged above the ledger.
3. **No schema change was needed.** Every `reply_log` row already carries its `ruleId` — the replier skills cite it on write (one row per action: sent/auto_sent/held/skipped) — so the ledger reads it straight from `snapshot("replies")`. The graphite/mint/cyan design system and the live-data hard requirement are unchanged.

What's new in v2.8

Per-run token telemetry, folded in per the living-spec rule. Neeraj saw his Muse subscription usage (347.6M tokens, almost all of it Sep 24–25 from building this harness) and asked for token visibility inside the dashboard: every run in the Runs tab must show its own token usage, with totals at the bottom.

1. **Measured, never estimated.** Before `run_close`, the coordinator (`run-coordinator` v1.3.0) measures the run's real token spend by summing the platform's own per-message metering (`agent.agent_message_token_usage` in `muse.db`) over its whole agent subtree — itself, every worker subagent it spawned (tracked from spawn results), and any grandchildren via `agent.agent_ancestors`. APPLY-phase browser driving is coordinator-level, so browser-task usage lands on the coordinator's own agent id and is included automatically. The totals are recorded with `token_record`, which now also persists `tokens_input` / `tokens_output` / `tokens_total` on the run row and takes a `usage_reported` flag (default true, backwards compatible). First verified measurement: the v2.8 telemetry build's own agent subtree used 5,287,537 input + 29,468 output tokens.
2. **Unreported is not zero.** If the metering rows haven't flushed yet when the run closes (they can lag a few minutes) or the metering DB is unreachable, the run is recorded with `usage_reported: false` and the Runs tab shows "—", never a fabricated 0. A run cannot genuinely use 0 tokens, so a bare 0 would be a lie. Every run closed before this change is unreported by construction — the only pre-existing numbers are the subscription-level daily aggregates on the customer's Muse usage page, which are account-wide, not per-run. Nothing is backfilled, ever.
3. **The Runs tab shows it.** A new "Token usage" column on every run row: compact total (19.5M / 104K / 860), full input/output numbers in the tooltip, per-stage breakdown (coordinator + one entry per worker skill, from the run's `tokens` map) expandable. A totals row is pinned at the bottom summing the visible runs — qualified "(reported runs only)" whenever any visible run is unreported. A per-day rollout line (America/Chicago) sits under the table. Graphite/mint/cyan, live data only, no sample numbers anywhere.

What's new in v2.9

Date-range filtering on the two busiest tabs, folded in per the living-spec rule. Neeraj asked for a date range across the dashboard so any slice of history can be isolated — starting with the ledgers he scans most.

1. **Replies tab: a date-range control that composes with everything.** Above the activity ledger, next to the channel/action dropdowns and the search box: preset buttons Today / Last 7 days / Last 30 days / All, plus a custom start/end date picker (native date inputs, America/Chicago calendar days, inclusive). The range filter composes with the existing channel, action, and search filters — a row shows only if it passes all of them. Default: All.

- 2. Runs tab: the same control, and the totals follow the filter.** Runs filter by started date (inclusive Chicago days); the pinned token totals row recomputes over the visible runs only — still qualified "(reported runs only)" when any visible run is unreported — and the per-day rollup line reflects the filtered set too.
- 3. No schema change, no new actions.** Both tabs already carry Chicago-localized timestamps in their snapshot rows; the filtering is pure client-side over the live rows the tabs already fetch. Graphite/mint/cyan, live-data hard requirement unchanged.

What's new in v2.10

The Schedules tab restores the drift-tracking view the old `scheduled-runs` page had. The consolidation had silently dropped it — today only a drift pill survives on each Runs-tab row — and the living-spec rule required the restoration to be folded back into the spec.

- 1. A seventh tab, read-only.** Schedules sits after Runs in the desktop rail and the mobile bottom nav. It reads the live `schedules_manifest.json` through a new published action, `schedules_status` (read-only): one row per job — job name, campaign, cadence, enabled state, last run time (America/Chicago), manifest body hash — plus a drift pill that reuses the existing Status tone logic (`drift` renders in the warn tone). Graphite/mint/cyan, deterministic Ask pattern, live-data hard requirement unchanged.
- 2. Drift derivation is honest about what the store knows.** Each campaign's drift comes from its latest run row's own `drift` value — the compiled-vs-live comparison the run-coordinator already performs at run time — with direct manifest-vs-live hash comparison as the fallback; `unknown` only when the campaign has no runs yet. A missing manifest produces an honest empty state ("No schedules manifest found — run compile-schedules"); no rows are ever invented.
- 3. No schema change, no other action touched.** `schedules_status` joins the manifest against the runs table. The rest of the action surface is unchanged.

What's new in v2.12

Two landed features, batched: a Profile tab UX overhaul, and a new `titles` profile field that drives scout query construction.

A. Profile tab UX overhaul. The Profile tab is now a settings-style page: a live summary header, clearer section hierarchy and field spacing, and explicit save-state feedback. The `profile_save` flow itself is unchanged — same validation, same file-first write contract. Two new elements: (1) every profile section carries an **eye-icon toggle** that reveals a SAMPLE JSON preview — realistic mock values plus allowed-value guidance (seniority `senior / staff / architect / principal / lead`, tiers 1–3, location priorities, yes/no screening answers, work-auth statuses), visually badged "SAMPLE" so it can never be mistaken for live data; mock data is confined to these previews, and the live-data rule holds everywhere else; (2) a **Schedules section** on the Profile tab lists all 12 jobs (job name, campaign, cadence, enabled state) through the read-only `schedules_status` action, with a jump link to the full Schedules tab for drift details.

B. `titles` — job titles become profile data, not worker invention. `targeting` gains a required `titles` field: a list of strings, minimum 1, free-form (no enum — titles vary too much to enumerate). The Targeting section renders it as an editable Job titles list (add/remove; an empty list is rejected on save), and the eye SAMPLE preview documents title examples plus how they feed query construction. The old "Role types" section is relabeled **"Employment type"**. In v2.13 it becomes an editable required multi-select over all five supported types; new profiles default to all five selected. Scout query construction is now deterministic: `job-board-search` queries and `linkedin-feed-hunting` Boolean blocks are built from `targeting.titles × seniority × industries`. The worker-never-invents-keywords hard limit is unchanged — if anything, it is now enforced by construction, since the keyword source is explicit profile data. `profile_save` requires `titles`, persists it to `profile.yaml` and the DB, and it round-trips through `profile_get`; a titles edit is a profile change, so `profile_watch` auto-reconciles it within ~15 minutes. Kit updates: `profile.schema.yaml`, both scout skills, and `profile.example.yaml` (template count unchanged at 13).

What's new in v2.13

Eight landed changes, folded in per the living-spec rule:

- 1. The dashboard is now Career Harness.** This is a display-name change only. Its technical artifact slug remains `harness-core` everywhere skills, schedules, actions, migrations, and operator documentation reference it.
- 2. Employment type is editable.** The Profile tab exposes `role_types` as a required multi-select: `full_time`, `part_time`, `w2_contract`, `c2c_contract`, and `internship`. New profiles select all five by default; the eligibility judge rejects postings whose type is not selected, and `profile_watch` recompiles after a toggle.
- 3. Every schedule can be edited in place.** All 12 Schedules rows now have an enable/disable switch and editable cadence. `schedule_update(job_id, cadence?, enabled?)` writes only that job's `campaigns` entry in `profile.yaml`; disabled jobs remain present with `enabled: false`. Unknown ids and unsupported cadence formats fail loudly.
- 4. YAML writes are type-safe.** The shared serializer quotes YAML-1.1-sensitive strings (`ok`, `yes`, `no`, `on`, `off`, case-insensitive) and strips server-shaped fields before writing the `answers` map.
- 5. Fresh installs start in the dashboard.** When `profile_get` returns null, Career Harness shows a six-step setup wizard instead of its seven tabs: identity, work auth, employment types, targeting/titles, screening answers, and caps. It reuses Profile components and `profile_save`; `[FILL IN]`-style placeholders fail on client and server. Success explains that `profile_watch` compiles schedules within about 15 minutes.
- 6. The Resumes tab manages the source library.** `resume_upload` validates `%PDF-` magic bytes, sanitizes and uniquifies the filename, computes SHA-256, collects variant metadata, and rejects duplicate `variant_id` values. `resume_delete` confirms exact usage, deregisters only the library row, and moves the file to recoverable trash; application history and tailored evidence are untouched. `file_open` can resolve a historical library file from trash.
- 7. PDFs open in the app.** Resume-library and per-application links open an in-page viewer with Close and Download controls, with a download fallback where embedded PDF rendering is unavailable.
- 8. Profile-watch has enough time to finish.** Its timeout is now 1,800 seconds (up from 900) after two otherwise-complete runs timed out during manifest rewrite; the rewrite now completes inside one run.

What's new in v2.11

Profile edits now recompile the schedules on their own. The old flow had a manual step nobody remembered: change the profile, then tell your Muse "recompile the schedules" — forget it and live schedules drifted from the profile while the doctor just reported the drift. A new ops job, `profile_watch` (every 15 minutes), closes that loop.

- 1. Detect via canonical hash, not file bytes.** The watcher parses `profile.yaml`, dumps it as JSON with sorted keys and compact separators, and sha256s that — comparing against the `profile_hash` in `schedules_manifest.json`. Raw file bytes were proven (in the dry run) to never match; YAML formatting churn is not a profile change.
- 2. Delta-only saves.** When the profile changed, the watcher runs the compile-schedules flow verbatim — validate, `profile_put`, render every template, then `cron.update` ONLY jobs whose rendered body actually changed (`cron.add` if a job is missing), verified back with `cron.view`. Unchanged jobs are never rewritten.
- 3. Loop guard.** If the profile changed but every rendered body is byte-identical, the watcher still stamps the new `profile_hash` into the manifest — otherwise it would recompile forever every 15 minutes.
- 4. Fail loud, never half-compiled.** Validation failure changes nothing: live schedules stay on the last good compile, an `event_log` row fires with the offending key, and the user gets a loud message. No change → silent exit (one debug line in the run record at most).

5. **Deliberate deviation:** the watcher's own body header omits `profile_hash` by design — stamping it would rewrite the watcher's own body on every profile edit and defeat the delta-only rule.
6. **The doctor is unchanged** — report-only drift backstop, as before. Its template line telling the operator to run compile-schedules manually is now stale: `profile_watch` handles it automatically (manual recompile remains as a fallback).

1. Architectural bets

1. **One store, enforced — not conventional.** Dedup, caps, and state transitions are unique constraints and guarded transitions inside `harness-core`, not sentences in a prompt. If the store refuses, the run stops.
 2. **Skills hold the logic; goals hold the contract.** A skill is pure instructions: Inputs, Actions called, Output (JSON, exact keys), Hard limits. No personal data — every skill reads customer facts from `profile_get` or rendered `{{profile.*}}` placeholders. A goal is a directory with a contract and a schedule that invokes skills. Fat skills, thin goals.
 3. **Hybrid storage, split by shape.** State, metadata, and hashes in SQLite. Blobs on the filesystem (PDFs, screenshots, JD snapshots), referenced by path + sha256. Never a PDF as a BLOB.
 4. **The schedule body is generated, never hand-written.** Rendered from templates + `profile.yaml` by the compile-schedules skill, validated against `profile.schema.yaml`, hash-stamped in `schedules_manifest.json`.
 5. **Intent before action.** An applier writes its intent (`applying` + `intent_id`) *before* clicking submit. Ordering, not polling, survives 30-minute handoff lags.
 6. **Gates by construction.** An applier task receives only an approved PDF path, one application row, and the standing answers. Unreviewed work is never handed over because it is never visible.
 7. **Silence by default.** Routine no-change runs are suppressed. The client hears what's new, risky, or requested.
 8. **Every run is resumable.** Rows keep their state in the DB; a crash never restarts from zero.
 9. **The kit is versioned software.** Semver, migrations, installer, smoke tests. Every run, event, and manifest row is stamped with the kit version that produced it.
-

2. Muse primitives — what each carries, and what we deliberately don't use

Primitive	Harness uses it for	Status
Scheduler (<code>cron.*</code>)	One compiled cron per campaign + <code>harness-doctor</code> ; bodies generated	Core
Main agent	Coordinator per run; report filter; only voice to the customer	Core
Subagents (<code>subagent.*</code>)	Scouts, judges, tailors — one skill per worker, small brief, max depth 2	Core
Browser tasks (<code>browser.*</code>)	One isolated Chromium task per application, per portal playbook	Core
Fullstack artifact	<code>harness-core</code> : the single store + callable API	Core
Skills (<code>~/workspace/skills</code>)	18 portable playbooks — the product's logic layer	Core
Goal directories	One per campaign; <code>hidden_files/</code> holds run scratch only	Core
Filesystem	Blobs, <code>profile.yaml</code> source, templates, manifests	Core
Memory (hot/warm/cold)	Narrative continuity for the main agent only — never a rule source	Core
Hooks (event-driven)	Pilot : inbox triggers (new mail/message → triage skill); cron watermark scan stays as backstop	Pilot in Phase 1
Workflows (deterministic JS)	Evaluate : coordinator reconcile loop as deterministic code instead of LLM orchestration	v2.3 candidate
Paired devices	Optional future : SMS/OTP codes with explicit per-use approval; default stays park-on-SMS	Not in core
Tracking, feed, ideas, media	—	Out of scope

Why hooks for inbox, and only as a pilot: the inbox is the one workload that is naturally event-driven — a new recruiter message exists, then we triage it. Hourly polling works (and remains the backstop with the watermark), but push cuts response latency from ~60 minutes to ~2. The pilot runs hooks alongside the hourly scan in shadow mode; if a hook misfires or drops an event, the watermark scan catches it. Polling is the safety net, not the enemy.

Why not workflows for the coordinator yet: the run-coordinator already exists as a well-specified skill, and the reconcile loop benefits from judgment calls (is this browser task stuck or just quiet?). A deterministic workflow would cut token cost and drift — but it also removes the judgment. v2.3 work, measured against real coordinator token spend first.

Hard constraints (unchanged): one customer = one Muse environment = one `harness-core`. No cross-environment state. Browser work only through browser tasks.

3. Skills-first architecture: thin goals, fat skills

This is the structural heart of v2.1. In v1, logic lived in three places — cron bodies, the main agent's memory, and convention. In v2.1, logic lives in exactly one place: the skill catalog.

3.1 The skill contract (every skill, no exceptions)

```
---
name: <skill-name>
description: <one line: what it does, when it runs>
---
# <Skill Name>
## Inputs          # exact JSON shape the worker receives
## Actions called # harness-core actions it may call (nothing else writes)
## Output          # JSON, exact keys – the verdict envelope
## Hard limits     # inviolable rules: what it must never do
```

Verdict envelope (all judges and staged skills):

```
{"skill": "<name>", "version": "1.0", "verdict": "pass|reject|hold",
  "score": 0-100, "reasons": [...], "evidence": {...}, "tokens": 1234}
```

Invocation contract — spawn a subagent with a short brief; the child inherits the transcript, so keep it small:

```
You are the <skill name>. Read ~/workspace/skills/<path>/SKILL.md and follow it
exactly. Inputs: <json>. Return ONLY the JSON verdict in the skill's Output
section. Hard limits in the skill apply.
```

Judges never write to `applications` directly — they return verdicts; the coordinator transitions. Browser skills run only through `browser.spawn_task`.

3.2 The catalog — 18 skills, 5 groups

Skill	Stage	Trigger	Writes via
linkedin-feed-hunting	Scout	linkedin_feed campaign	posting_upsert, event_log
job-board-search	Scout	job_board campaign	posting_upsert
career-portal-sweep	Scout	career_portal campaign	posting_upsert, companies
eligibility-judge	Screen	every new posting	verdict only
h1b-judge	Screen	every new posting	h1b_lookup, verdict
fit-judge	Screen	eligibility pass	verdict only
resume-picker	Pick	screened	resume_pick, verdict
resume-tailor	Tailor	resume_picked	file + hash on row
resume-reviewer	Review gate	tailored	review_get/put
screening-answerer	Apply	per form question	approval_enqueue
portal-navigator	Apply	reviewed	app_transition (intent, submitted)
email-replier	Conversations	email_scan campaign (Outlook via outlook-mail)	conversation_upsert, reply_log
linkedin-replier	Conversations	linkedin_replies campaign	conversation_upsert, reply_log
approval-judge	Queue	every 2h + on demand	approval_resolve
run-coordinator	All	each campaign cron	run_open/close, app_claim, app_transition
compile-schedules	Ops	install + rule change	cron.update, event_log
harness-doctor	Ops	hourly	event_log, verify-only tasks
daily-report	Ops	daily	snapshot (read only)

The `portal-navigator` carries per-ATS playbooks (`playbooks/workday.md`, `greenhouse.md`, `lever.md`, `ashby.md`, `icims.md`, `easy-apply.md`, `generic.md`) — portal quirks are data files under the skill, not branches in a prompt.

3.3 Shared conventions (in the skills README, enforced by review)

- **Hashes:** `jd_hash` = sha256(normalized JD text)[:12]; `resume_hash` = sha256(PDF bytes)[:12].
- **Normalization:** `company_norm` = lowercase, strip legal suffixes (inc, llc, corp, ltd), collapse spaces; `role_norm` = lowercase, strip seniority adjectives and req ids.
- **Never invent:** any fact not in `profile_get` or the resume variant → `hold`, never a guess.
- **Voice** (conversations): 2–4 short sentences, plain words, like a person replying from a phone. Never em dashes, never "hope this finds you well", never resume PDFs on LinkedIn.
- **Every skill appends one `event_log` row on exit** with its verdict and token count — observability is not optional.

3.4 What a goal looks like now (thin by design)

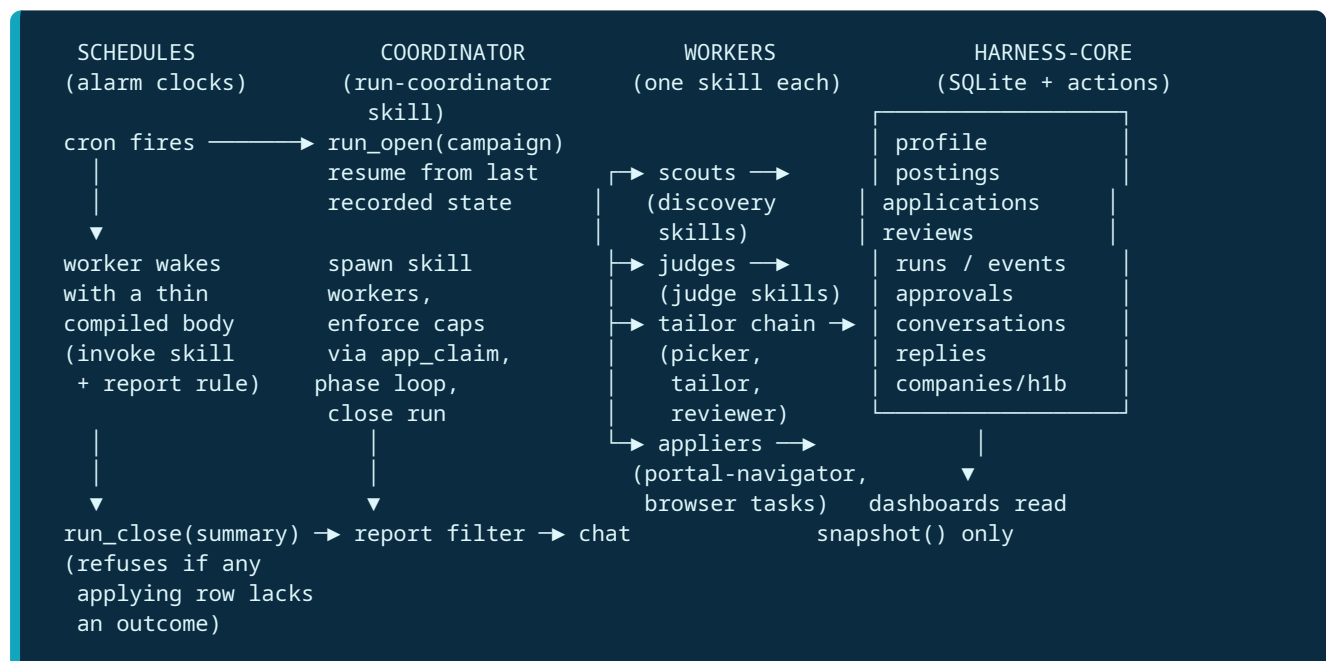
```
goals/<campaign>/
├── GOAL.md           # the contract: what this campaign does, its caps, its skill chain
├── crons/            # read-only copies of the compiled body (the saved body is truth)
├── files/            # documents the CUSTOMER asked for (Goals tab)
└── hidden_files/<run>/ # run scratch: tailored PDFs, JD snapshots, screenshots,
                        # spillover.jsonl – evidence, never state
```

A goal contains **no logic**. Its cron body is ~30 lines: invoke the run-coordinator skill with `campaign_id`, close the run, apply the report rule. Everything the campaign *does* is a skill invocation. This is what makes "runs on each system" true: installing a campaign is copying a directory and compiling one body.

3.5 Skill versioning

Each skill's frontmatter carries a version. The kit version (semver) stamps every run, event, and manifest row. When a skill improves, the kit version bumps, `compile-schedules` re-renders, and the manifest records which skill versions produced which bodies. A customer asking "why did last Tuesday behave differently?" gets an answer from the manifest — not archaeology.

4. System overview — four layers



The coordinator *is* the run-coordinator skill: it opens the run, resumes in-flight rows from `harness-core`, pipelines the skill chain with parallel workers, enforces caps through `app_claim`, closes the run, and reports only what's new. Workers never message each other; they transact with the store and return verdict JSON.

5. Career Harness (harness-core): the single store

One fullstack artifact, displayed as **Career Harness** and addressed technically by the unchanged slug `harness-core` (SQLite via Drizzle). Every worker and the coordinator touch it **only through published actions**.

5.1 Schema — 14 tables

Table	Key	Holds
profile	singleton	Work auth, role types, locations, tiers, industries, comp handling, start date, standing answers, caps, reply tiers
resume_variants	variant_id	Path, sha256, role family, industry tags, years matrix, keyword vector, times picked, approval rate
campaigns	campaign_id	Type, cadence, cap per run/day, gates, source list, cron id
postings	posting_id (sha of company+role+url)	Company, role, url, source, jd_path, jd_hash, first_seen, last_seen
applications	app_id; unique (company_norm, role_norm)	State, campaign, variant, resume_hash, intent_id, evidence path, blocker, outcome, timestamps, kit_version
reviews	(jd_hash, resume_hash)	Verdict, notes, reviewer version, decided_at
runs	run_id	Campaign, kit_version, started, ended, status, counts, tokens, tokens_input, tokens_output, tokens_total, tokens_reported, needs_me
events	autoincrement	Append-only: run_id, app_id, type, payload, at. Every transition writes one row first, in the same transaction
approvals	approval_id	Queue item: kind, app_id, question, options, judged_by, resolved_at, answer
h1b_sponsors	company_norm	USCIS approvals/denials by fiscal year, LCA count, last refreshed
companies	company_norm	Tier, industry, HQ state, careers url, ATS type, park count, skip flag + reason
contacts	contact_id	Recruiter / hiring-manager pages
conversations	thread_id (channel + external thread key)	Channel, contact_id, app_id if matched, classification, state, watermark
replies	reply_id	thread_id, direction, action (sent held auto_sent skipped), tier + rule id, draft path, reason, run_id, approval_id

5.2 Constraints that do the enforcing

- **Dedup:** unique (`company_norm`, `role_norm`) on `applications`. `app_claim` refuses duplicates — double-applying across campaigns is *impossible*, not unlikely.
- **Caps:** `app_claim` counts per run and per day in SQL and refuses at the cap. No prompt-counting.
- **Transitions:** `app_transition` accepts only legal edges (§10) and writes the `events` row in the same transaction.
- **Review cache:** unique (`jd_hash`, `resume_hash`) on `reviews` — the gate is a lookup.
- **Terminal states** (`submitted`, `confirmed`, `blocked`, `rejected`) are never re-entered. Re-runs are safe by construction.

5.3 What stays on the filesystem (and why)

On disk	Referenced from DB by
Resume variant source PDFs (<code>user/files/</code>)	<code>resume_variants.path</code> + sha256
Tailored PDFs (<code>goals/<g>/hidden_files/<run>/resumes/<slug>/</code>)	<code>applications.resume_hash</code> + path
JD snapshots	<code>postings.jd_path</code> + <code>jd_hash</code>
Screenshots (every apply step + confirmation)	<code>applications.evidence_path</code> , <code>events.payload</code>
<code>profile.yaml</code> (validated screen mirror; operator-readable)	<code>profile_put</code> compiles it into the <code>profile</code> table; both must agree
<code>templates/<campaign>.body.md</code> + <code>profile.schema.yaml</code>	rendered into cron bodies by <code>compile-schedules</code>
<code>schedules_manifest.json</code> (job id, campaign, cadence, body hash, skill versions, <code>profile_hash</code> , <code>compiled_at</code>)	pushed via <code>event_log</code> ; dashboard shows compiled-vs-live drift
Per-run spillover log (<code>hidden_files/<run>/spillover.jsonl</code>)	replayed via <code>spillover_replay</code> if the artifact is unreachable

Rule of thumb: bigger than a few KB or streamed by a worker → file. A decision depends on it → row. Logic → skill.

6. Published actions API

Action	Called by	Contract
profile_get	every worker at start	Returns the profile as JSON. Replaces reading <code>answers.md</code>
profile_put	compile-schedules	Syncs validated <code>profile.yaml</code> into the <code>profile</code> table; returns <code>profile_hash</code> . Bodies and DB can never disagree
posting_upsert(rows)	scout skills	Dedups on <code>posting_id</code> and <code>(company_norm, role_norm)</code> ; returns only genuinely new ids
app_claim(posting_id, campaign_id)	run-coordinator	Creates <code>discovered</code> or refuses (cap, duplicate, skip-listed). The limiter
app_transition(app_id, from, to, evidence, intent_id?)	every stage	Rejects illegal transitions; writes the <code>events</code> row in the same transaction
review_get / review_put	resume-reviewer	Cached verdicts on <code>(jd_hash, resume_hash)</code>
resume_pick(jd_text, role_family?)	resume-picker	Top 3 variants with scores and reasons
h1b_lookup(company)	h1b-judge	Sponsor record + score 0–100 + evidence years
companies_update	portal-navigator, scouts	Corrects <code>ats_type</code> , increments <code>park_count</code> — the only writer to <code>companies</code> besides imports
approval_enqueue / approval_resolve	judges, approval-judge, coordinator	The single review queue
run_open / run_close	run-coordinator	Bookends. <code>run_close</code> refuses if any application is still <code>applying</code> without an outcome
event_log(run_id, type, payload)	every skill on exit	Verdict + token count; free-form observations
snapshot(view)	dashboard tabs, daily-report, doctor	Read-only aggregates: <code>overview</code> , <code>applications</code> , <code>resumes</code> , <code>runs</code> , <code>replies</code> , <code>health</code> , <code>ask</code> . The only read API the UI uses
h1b_import / companies_import	installer	Bulk loads from shipped CSVs
conversation_upsert / reply_log	replier skills	Every inbound message and every sent/held reply; watermark advances in the same transaction
spillover_replay(run_id)	doctor	Idempotent replay of a run's local spillover log after an outage
test_data_purge()	repair jobs, operator	Deletes ONLY strict-marker smoke/test rows (applications, resumes, runs, profiles + smoke-marked dependents). Fail-closed: refuses entirely if any candidate or dependent lacks a marker; never cascades silently; never touches <code>event_log</code> ; one audit event lists every deleted id. Idempotent

Action	Called by	Contract
<code>evidence_attach(app_id, resume_path, resume_hash, variant_id, screenshot_path?, confirmation?)</code>	portal-navigator, run-coordinator, repair	Writes the submission evidence bundle onto an application row. <code>resume_hash</code> schema-enforced to 64-char lowercase hex; <code>screenshot_path</code> may be null (explicit "not captured"). Never transitions state
<code>file_open(app_id, kind) / file_open(variant_id)</code>	dashboard tabs	Record-bound file opener: <code>kind</code> is <code>resume</code> , <code>screenshot</code> , or <code>confirmation</code> . Resolves only the path recorded on that row (or the registered variant); if a deregistered library PDF is referenced by history, it falls back to that variant's recoverable trash path. Arbitrary paths are refused
<code>profile_save(profile)</code>	Profile tab, setup wizard	Validates on client and server, rejects placeholder values, serializes safely, writes <code>profile.yaml</code> first, then <code>profile_put</code>
<code>schedules_status()</code>	Profile, Schedules	Returns all 12 manifest jobs with cadence, enabled state, last run, body hash, and drift
<code>schedule_update(job_id, cadence?, enabled?)</code>	Schedules tab	Validates the id and cadence, then surgically updates only that job under <code>campaigns</code> in <code>profile.yaml</code> ; disable writes <code>enabled: false</code> , never deletion
<code>resume_upload(pdf, metadata)</code>	Resumes tab	Validates %PDF - magic bytes; sanitizes and uniquifies the filename under <code>workspace/user/files</code> ; computes SHA-256; rejects duplicate <code>variant_id</code> values (no upsert)
<code>resume_delete(variant_id)</code>	Resumes tab	Returns exact usage for confirmation, deregisters only the library row, and moves its file to recoverable trash; application rows and tailored evidence are immutable

Privileged file contracts. `writeProfileYaml` is the existing allowlisted writer. v2.13 adds `writeResumeFile`, `trashResumeFile`, and `readTrashedResume`, each path-scoped to the resume library or `~/workspace/.trash/resumes/`; none accepts arbitrary paths.

7. Profile as data, cron bodies compiled

A rule is written **once**. Fresh installs collect it in Career Harness's setup wizard; after first setup it is maintained on the **Profile tab** (section 13), which writes back to both the live store and the YAML file. The compile-schedules skill generates every cron body from the same facts.

7.1 `profile.yaml` (shape — validated mirror of the screen)

```
identity:  {name, email, phone, city, state, linkedin_url}
work_auth: {status: H-1B, sponsor_required: true, h1b_gate: soft} # hard | soft
role_types: [full_time, part_time, w2_contract, c2c_contract, internship] # min 1; all five
selected by default
locations: {us_only: true, remote: ok, metros: [Houston, Dallas, Austin]}
targeting: {tiers: [1, 2], industries: [fintech, healthcare], seniority: [senior, staff],
titles: ["Senior Data Engineer", "Staff Data Engineer"]}
# titles: required, list of strings, min 1, free-form (no enum — titles vary too
much to enumerate).
# Drives scout query construction: job-board-search queries and linkedin-feed-
hunting Boolean blocks
# are built from titles × seniority × industries (worker-never-invents-keywords, by
construction).
comp:      {floor: null, negotiable_answer: "Negotiable", zero_ok: true}
start_date: 2026-10-09
answers:   {relocate: No, restrictive_covenants: Yes, drivers_license: Yes, degree_dates:
decline}
caps:      {per_run: 10, per_day: 30, appliers: 2, linkedin_actions_per_hour: 20}
reply_tiers: {auto_send: [R1, R3, R5], draft_for_review: [comp_questions, new_contacts],
never: [contract_pitches]}
resumes:   {dir: user/files, filename_rule: "<Name>_Data_AI_Resume.pdf"}
campaigns: {linkedin_feed: {cadence: 2h weekdays}, career_portal: {cadence: daily 14:20},
job_board: {cadence: nightly 23:20}}
```

7.2 compile-schedules (the skill)

1. Validate `profile.yaml` against `templates/profile.schema.yaml` (required keys, enums). **Fail loudly** on a missing key — never render a half-profile.
2. `profile_put` — the DB and the bodies are rendered from the same facts.
3. Render each `templates/<campaign>.body.md` (`{{ }}` placeholders). **A placeholder with no value fails the compile** — `[FILL IN]` never ships.
4. Bodies are thin: invoke the run-coordinator skill with `campaign_id`, then the standard footer (run_close, report rule).
5. `cron.update` ONLY for jobs whose rendered body actually changed (compared against `cron.view` of the live body); `cron.add` if a job is missing; disabled campaigns are saved with `enabled: false`, **not deleted** — re-enabling is a recompile, not archaeology.
6. `cron.view` each saved/updated job back; sha256 each saved body; write `schedules_manifest.json` (job id, campaign, cadence, hash, skill versions, profile_hash, compiled_at).
7. `event_log` the manifest. **A rule stated in chat is not live until this skill runs** — say so, every time (the `profile_watch` ops job, section 7.4, runs it automatically within ~15 min of a profile change; a manual run remains the fallback).

Hard limit: the `profile_watch` ops job auto-recompiles within ~15 min of any `profile.yaml` change; the doctor remains report-only as the drift backstop.

7.3 The Profile tab — runtime profile management with two-way YAML sync (UX overhaul, v2.12)

`profile.yaml` is the seed; the Profile dashboard tab is the ongoing control. The tab loads the profile with `profile_get` and is now a settings-style page: a live summary header, clearer section hierarchy and field spacing, and explicit save-state feedback. The underlying `profile_save` flow is unchanged — same validation, same file-first write contract (below).

Sections render as editable fields (identity, screening answers incl. add-new-answer, caps, targeting, locations, work_auth, comp, start_date, reply_tiers, employment types). The targeting and employment-type controls are:

- **Targeting** now includes an editable **Job titles** list (add/remove). It maps to the required `targeting_titles` field (list of strings, min 1, free-form, no enum); an empty list is rejected on save, client-side and by `profile_save`.
- **Employment type** is an editable required multi-select mapped to the unchanged YAML key `role_types`. Allowed values are `full_time`, `part_time`, `w2_contract`, `c2c_contract`, and `internship`; at least one is required, and a fresh profile defaults to all five. The eligibility judge rejects any posting whose employment type is not selected. Saving a toggle flows through `profile_watch` like every other profile change.

Every profile section carries an **eye-icon toggle** that reveals a SAMPLE JSON preview: realistic mock values plus allowed-value guidance (seniority `senior / staff / architect / principal / lead`, tiers 1–3, location priorities, yes/no screening answers, work-auth statuses), visually badged "SAMPLE". Mock data is confined to these previews — the form fields themselves show only live profile values, and the live-data hard requirement holds everywhere else.

A **Schedules section** on the Profile tab lists all 12 jobs (job name, campaign, cadence, enabled state) through the read-only `schedules_status` action (same data path as the Schedules tab — no new action, no new data path), with a jump link to the full Schedules tab for drift details.

`profile_save` performs the save as one verified unit:

1. Validate all fields (same rules as `profile.schema.yaml`: required keys, types, enums, integer minimums).
2. Render the YAML text from the profile (reverse of the compile mapping). The file's header comment is preserved; the `resumes` block and any untouched `campaigns` entries are carried over verbatim. Key renames are handled explicitly (`covenants` ↔ `restrictive_covenants`; `location: "City, ST"` ↔ `city/state`; `linkedin` ↔ `linkedin_url`; `note` ↔ `negotiable_answer`). The shared serializer quotes YAML-1.1-sensitive scalar strings (`ok`, `yes`, `no`, `on`, `off`, case-insensitive) and zip-like strings, so they are never coerced to booleans or integers. Before serialization, `answers` is rebuilt from its allowlisted answer keys; server-shaped fields never leak into that map.
3. Re-parse the rendered YAML and validate it against the schema rules again (read-back check). On failure: change nothing, return the offending field.
4. Write `~/workspace/profile.yaml` FIRST via the allowlisted privileged file-write contract (`writeProfileYaml` — exactly one permitted path, realpath-verified, no traversal), then `profile_put`. File-first ordering means a DB failure self-heals: the next `compile-schedules` run re-syncs YAML → DB and they converge. Either step failing returns an error naming the exact step; the UI shows success only when both succeed.
5. The `locations` mapping (DB `priority / relocation` ↔ YAML `us_only / remote / metros`) is lossy by construction; the action applies a documented derivation and, for any priority entry it cannot map safely, keeps the previous YAML block and surfaces a visible notice naming the field. No field is ever silently skipped.

Because the YAML always mirrors the screen, `compile-schedules` keeps working unchanged: it validates the file, syncs it into the store, and recompiles bodies — a no-op when nothing drifted.

7.4 `profile_watch.body.md` — auto-recompile (v2.11)

The 12th scheduled job (`harness-profile-watch`, every 15 minutes, America/Chicago; timeout 1,800 seconds) is the only automatic writer of schedule bodies. The timeout was raised from 900 seconds after two runs completed schedule reconciliation but timed out at manifest rewrite; the longer window lets the manifest finish in the same run. It replaces the old manual "recompile the schedules" step.

Detect. Parse `~/workspace/profile.yaml` as YAML; canonicalize to a JSON dump with sorted keys and compact separators (`,`, `:`), UTF-8; sha256 → `current_hash`. Compare against `schedules_manifest.json`'s `profile_hash` (missing manifest = first run → recompile). Hashes match → silent exit: no `event_log` row, no user message. **Never compare raw file bytes** — the dry run proved formatting churn never matches; only semantic profile changes count.

Recompile (hashes differ). Follow the compile-schedules skill verbatim, with watcher-specific rules:

1. Validate against `templates/profile.schema.yaml`. On ANY failure: change NOTHING — live schedules stay on the last good compile. `event_log` loudly (naming the failing key) AND surface a message to the user. Never half-compile.
2. `profile_put` → new `profile_hash`.
3. Render every `templates/<campaign>.body.md` (including this one).
4. `cron.view` each live job; `cron.update` ONLY jobs whose rendered body actually changed; `cron.add` if a job is missing. Never delete a job; disable via `enabled: false`.
5. `cron.view` to verify every saved body; recompute body hashes per the manifest's hash scheme (sha256 of the rendered body with the `# body_hash:` field blanked).
6. Rewrite `schedules_manifest.json`: all jobs, new `profile_hash`, new `compiled_at`, a note naming `old_profile_hash` → `new_profile_hash` and which jobs changed.
7. **Loop guard:** if the profile changed but every rendered body is byte-identical to the live bodies, STILL write the new `profile_hash` into the manifest — otherwise the watcher recompiles forever every 15 minutes.
8. `event_log` one row: old → new profile hash, jobs changed (or "none — profile-only change"), verdict.

Report rule: no change → silent; recompiled → one concise message naming which schedules changed, if any; validation failure → loud message + `event_log`, live schedules untouched.

Deliberate deviation: this ops job's body header intentionally omits `profile_hash`. Its logic only *detects* profile changes; stamping the hash in its own header would rewrite its own body on every profile edit and defeat the delta-only update rule.

8. Campaigns and the schedule fleet

One campaign = one thin goal + one compiled cron. All campaigns run the same seven-stage skill chain; they differ only in the scout skill and the portal playbook.

Campaign	Scout skill	Apply path	Default cadence (CT)
linkedin_feed	linkedin-feed-hunting (Boolean blocks, past-24h, US-only)	Easy Apply playbook	every 2h, weekdays
career_portal	career-portal-sweep (companies by tier/industry → careers pages, ATS detection)	Workday / Greenhouse / Lever / Ashby / iCIMS playbooks	daily 14:20
job_board	job-board-search (query builder + ranking)	Board apply or redirect into a portal playbook	nightly 23:20
email_scan	Outlook mailbox via the outlook-mail skill, since conversations watermark (Gmail is never used — the Gmail connector is not connected)	email-replier skill	hourly, weekdays
linkedin_replies	LinkedIn inbox via browser task since watermark — bounded: unread filter first, max 10 threads/run, 15-min timebox (stop and report even if incomplete)	linkedin-replier skill	hourly, weekdays

Scout query construction — deterministic from `targeting_titles` (v2.12)

`job-board-search` queries and `linkedin-feed-hunting` Boolean blocks are built from `targeting_titles × seniority × industries` — the Cartesian product of the profile's title list, seniority list, and industry list, rendered into query strings and Boolean blocks by the scout skills. Titles are free-form profile data (no enum), so the worker never invents keywords: the query vocabulary comes from the profile or it doesn't ship. The worker-never-invents-keywords hard limit is unchanged — it is now enforced by construction rather than by instruction alone. A titles edit flows through `profile_save` → `profile.yaml` → `profile_watch` auto-recompile, so the next scout run after the change already searches the new titles.

Control: `approval-judge` every 2h · `daily-report` 9:00am · `token-usage` 9:20am · `weekly-review` Fri 5:00pm · `harness-doctor` hourly · `profile_watch` every 15 min (auto-recompile on profile change, section 7.4). Weekday-only jobs exit silently on weekends; Monday resumes from the unchanged watermark.

The run workflow — 7 stages, pipelined, one skill per stage

SCOUT → SCREEN → PICK → TAILOR → REVIEW GATE → APPLY → VERIFY
(skills) (judge (resume- (resume- (resume- (portal- (24h email
 skills) picker) tailor) reviewer) navigator) match)

Pipelined, not batched: stage n+1 starts on the first row, not the last. Scouts still searching while the first resume is being tailored; applicers submit each resume the moment its approval lands. Same guarantees as strict sequencing, much shorter wall-clock.

- **Scout** — scout skills write via `posting_upsert`; the action drops known/skip-listed rows, so tokens are never spent twice.
- **Screen** — eligibility → h1b + fit in parallel (judge skills, verdict JSON). Hard fail → `rejected` with reason. Pass → `app_claim` (caps enforced in SQL).
- **Pick** — `resume-picker` returns top 3 with scores; top 1 used, others logged for tuning.
- **Tailor** — `resume-tailor`: bounded edits only; output per the strict filename rule; sha256 on the row.

- **Review gate** — `resume-reviewer: review_get` first; on miss run the skill, `review_put` caches. Rejected → `parked`, never downstream.
- **Apply** — `portal-navigator` via `browser.spawn_task`, given only `app_id`, `url`, `ats_type`, `pdf_path`, `resume_hash`. Intent before submit; hash re-verification; screenshots per page; unknown question → `approval_enqueue`; CAPTCHA/SMS → `parked` with checkpoint.
- **Verify** — `submitted` only on a captured confirmation string; optional 24h email match in the Outlook mailbox → `confirmed`.

The run ends when scouts are done and no row sits in `discovered...applying`. `run_close` writes counts; the coordinator reports **only if something new happened**.

9. Application state machine and the intent log

`app_transition` accepts only these edges. Anything else is refused and logged. Terminal states are never re-entered — except `blocked → discovered`, which exists solely as the audited coordinator-correction path above.

From	To	Who	Evidence required
—	discovered	app_claim	posting_id, campaign, cap check passed
discovered	screened / rejected	judge skills	verdict rows for all three judges
screened	resume_picked	resume-picker	variant_id + score
resume_picked	tailored	resume-tailor	pdf path + sha256
tailored	reviewed / parked	resume-reviewer	reviews row
reviewed	applying	portal-navigator	intent_id, written before the submit click
applying	submitted / blocked / parked / needs_me	portal-navigator	evidence bundle via <code>evidence_attach</code> : confirmation text + screenshot path (or explicit "not captured") + exact uploaded resume path + full sha256; or blocker + checkpoint
submitted	confirmed	verifier (24h)	email message id
needs_me	reviewed	approval_resolve	answer recorded in profile answers
parked	reviewed	coordinator retry, max 2	checkpoint path
blocked	discovered	coordinator correction only	<code>evidence.coordinator_correction=true</code> + <code>event_log</code> id of a <code>coordinator_correction</code> event for that app + reason; re-screened from the ledger row by the next run

Correction precedent (2026-09-26): four morning-run postings (Akumin, Step, CVS Health, Stryker) were blocked on invalid verdicts — applier briefs used stale summary URLs that 404'd while the ledger URLs were live. Each got its own `coordinator_correction` event (ids 270–273, referencing the worker's correction log id 247) and was re-opened `blocked` → `discovered` through the audited edge above. The rows re-enter the pipeline at `discovered`; nothing was resubmitted and no other `blocked` row was touched.

The intent log

Owned end-to-end by the portal-navigator skill:

1. Write `applying` with a fresh `intent_id` and the exact URL — refused if the row isn't `reviewed` (double-claim guard).
2. Click submit.
3. Write `submitted` only after the evidence bundle is complete: `evidence_attach` with the confirmation text, the screenshot path (or `null` with an explicit "not captured" marker — a lost capture is recorded honestly, never faked), the exact uploaded resume path, and its full 64-char sha256. The coordinator verifies every file exists before the transition is written. The dashboard's evidence links open these files through `file_open` (section 13.7).

If step 3 never arrives, the row sits in `applying` with an unresolved intent. The doctor finds it after 90 minutes and spawns a **verify-only** task (portal's applied-jobs list or confirmation email), transitioning to `submitted` or back to `reviewed`. **Nothing is ever re-submitted while an intent is unresolved.** Portal quirk discovered mid-run ("already applied" banner) → `blocked` reason `duplicate_portal` with screenshot — the store learns what the scout missed.

10. Judges and scoring

Judges are cheap, parallel, one-skill subagents returning the verdict envelope. Scoring math lives in `harness-core` actions — deterministic, tunable without touching prompts.

Judge / algorithm	Input	Output	Rule
eligibility-judge	JD text + <code>profile_get</code>	pass / reject + reason	Posting employment type must be selected in <code>profile.role_types</code> ; unselected types reject. US/remote per profile; agency posts rejected unless C2C is selected
h1b-judge	<code>company_norm</code>	score 0–100 + evidence	<code>h1b_lookup</code> ; <code>hard</code> → reject below 40; <code>soft</code> → rank down; unknown → careers-page check, logged
fit-judge	JD text + targeting	0–100	Seniority, industry, must-haves vs years matrix; below threshold → reject. Never invents years; null/absent matrix → <code>hold</code> ("cannot score fairly"), never a mass-reject
resume-picker	JD text, role family	top 3 + scores	$0.45 \cdot \text{keyword} + 0.25 \cdot \text{tag match} + 0.20 \cdot \text{historical approval rate} + 0.10 \cdot \text{variant recency}$
resume-reviewer	tailored PDF + JD	approved / approved-with-notes / rejected	Cached on (<code>jd_hash</code> , <code>resume_hash</code>); banned-phrase check; truthful years matrix
approval-judge	<code>approvals</code> queue	auto-resolved or batched	Risk rubric; never auto-answers a question not in answers; creative content never auto-approved
source ranking	<code>companies</code> tiers/industries/metros	ordered sweep list	Tier 1 in target industry first; skip-flagged never fetched

Every verdict + reasons lands in `events`. "Why did you skip Stripe?" is answered from the log, not reconstructed.

H-1B data: `h1b_employer_hub.csv` (USCIS Employer Data Hub, FY2021–2025) + LCA summary ship in the kit; `h1b_import` at install; quarterly `runonce` refresh. The H-1B judge is a lookup, not a guess.

11. Inbox: conversations and the replies ledger

The email-replier and linkedin-replier skills share one playbook (rule ids R1–R8), so both channels behave identically:

Mailbox wiring (email channel). The email-replier reads and replies exclusively through the `outlook-mail` skill against the customer's connected Outlook mailbox (`neerajreddyr@outlook.com`). The compiled `email_scan` cron body names `outlook-mail` explicitly and forbids Gmail — the Gmail connector is not connected and must never be used for scanning or sending. Everything else about the email channel is unchanged: same tiers, same watermark rule, same report rule.

Rule	Situation	Reply shape
R1	Recruiter outreach, role fits	Thanks, interest, sponsorship line if required, ask for JD or quick sync
R2	Outreach, role doesn't fit (contract/C2C)	One-line polite decline, open to full-time
R3	Interview invite	Enthusiasm + availability windows; never a single binding time
R4	Resume request	Confirm attached (email only), filename per rule
R5	Human-written application update	Short acknowledgment
R6	Follow-up nudge (≥7d email / ≥4d LinkedIn quiet)	One-line check-in, no attachments
R7	Marketing / sales pitch	"Not right now" deferral
R8	Auto-rejection / system mail	Log outcome, no reply

HOLD regardless of tier: salary numbers, specific call times, exclusivity, documents beyond resume, sensitive-data asks, scam signals, anything outside R1–R8.

LinkedIn inbox scan bounds. The `linkedin_replies` browser pass is bounded so one heavy inbox can never stall the hour: unread filter first, at most 10 threads per run (newest-first since the `conversations` watermark), and a 15-minute timebox — stop and report even if threads remain. Partial coverage is normal; the run closes with `threads_triaged` and `threads_skipped_timebox` counts. The watermark advances only over threads actually triaged (a reply row written), never over threads cut off by the timebox.

Per inbound message: **classify** (rejections/interviews set `applications.outcome` on match) → **route by tier** (`auto_send` / `draft_for_review` / `never`) → **act** (one `replies` row per action: `sent` / `auto_send` / `held` / `skipped`, with the rule id cited) → **advance the watermark only after the row is written**. Every auto-sent reply cites its rule id — the approval judge audits the citations.

Rules encoded as data: `reply_log` refuses `.pdf` attachments on `linkedin`; do-not-email contacts refuse `sent`. The Replies Ledger dashboard reads `snapshot("replies")`: threads by state, replies/week, outcome funnel (applied → reply → interview), and the awaiting-me list.

12. Approvals and safety

Two independent layers:

- 1. System permission cards (runtime):** spending, credentials, sensitive access. Only the customer's tap approves. No agent or operator can approve on their behalf.
- 2. Harness send tiers (compiled bodies):** `auto_send` for routine replies under the playbook; `draft_for_review` for everything else. The approval-judge sweeps every 2h and batches one go-ahead. "Wait" is a hard stop on everything, mid-flight.

Honesty rules (every body, no exceptions): never invent screening answers or years — profile is the only source; every tailored resume clears the reviewer; uploaded file = reviewed file (hash-verified); banned phrases never ship unverified; no drip-fed micro-questions — unanswerable items batch into one ask. The screening-answerer skill turns "unknown" into `approval_enqueue`, never a guess.

13. Career Harness — one artifact, seven tabs, focused write APIs

The old fleet was five separate artifacts (command-center, command-center-hub, scheduled-runs, linkedin-posts, replies-ledger), each with its own SQLite database. The old code taught us two things that now shape this design:

- 1. Cross-artifact action calls don't work.** The hub's DATA-PLAN documents it: from inside an artifact's server, `artifact.invoke_action` cannot target a sibling artifact — the hub fell back to privileged filesystem reads of sibling DB files. Any design with "dashboard artifacts calling harness-core actions" is broken by construction. The dashboards must live *inside* `harness-core` as tabs, calling `snapshot()` as same-artifact actions.
- 2. The divergence wart was topological.** Five databases plus CSVs plus merge rules produced the 37-vs-25 split. One artifact, one database, one read API — the wart becomes unrepresentable.

So **Career Harness** serves the UI from the unchanged `harness-core` artifact slug. The old fleet collapses into seven tabs (linkedin-posts is dropped — Presence/Content are out of scope):

Tab	Replaces	Shows
Overview	command-center-hub	Cards per area with live headline numbers, fleet health 24h, approval batch, Ask
Applications	command-center	KPI band + full ledger with evidence columns (resume file, sha256, screenshot status, confirmation)
Resumes	command-center Resume Library	Source-PDF library with exact usage, upload, recoverable delete, and in-page PDF viewing
Runs	scheduled-runs	Run history, Token usage column (compact in/out/total per run, totals row, per-day rollup), per-stage tokens, blockers, compiled-vs-live drift; date-range filter (v2.9), totals recompute over the filtered set
Schedules	scheduled-runs	Per-job drift ledger plus v2.13 enable/disable and cadence editing: live status reads through <code>schedules_status</code> ; changes use <code>schedule_update</code>
Replies	replies-ledger	Unified chronological activity ledger (v2.7): Date · Channel · Person/Topic · Action · Rule (R1–R8 pill) · Reason/Source, email + LinkedIn in one timeline, newest first; date-range filter (v2.9) composing with channel/action/search; header keeps sent/held counts, outcome funnel, awaiting-me
Profile	(new, v2.4)	Live profile editor — every section editable, saves via <code>profile_save</code> (DB + YAML two-way sync)

13.1 `snapshot(view)` — the only read API

View	Returns
overview	KPI cards: submitted today (Chicago time), submitted 7d/30d, replies this week, needs-you count; fleet health per campaign 24h; approval batch
applications	Full ledger rows (date, company, role, campaign, state, blocker, link, jd_hash, variant_id, evidence path) + evidence bundle per row (uploaded resume file, full sha256, screenshot status, confirmation text — each opened via <code>file_open</code> , section 13.7) + KPIs: total, submitted, blocked, rejected, parked
resumes	Registered variants (filename, path, sha256, role family, industry tags, years matrix, keyword vector, size, mtime) + exact application usage counts + record-bound PDF open state
runs	Run history (run_id, campaign, kit_version, started/ended, status, counts, tokens, tokens_input, tokens_output, tokens_total, tokens_reported, blockers, needs_me) + per-stage token breakdown; unreported runs show "—", never 0
replies	Unified activity ledger rows (at, action, direction, reason, ruleId, threadId, replyId, runId) — newest first, both channels — plus threads by state, outcome funnel (applied → reply → interview), awaiting-me list
health	Doctor's latest output: status, drift rows, stale intents, aged approvals, H-1B data age, disk usage
ask	Deterministic Q&A over the live tables (see 13.4)

13.2 Schema refinements (from the old `schema.ts`)

- `applications` keeps the old (`company_key`, `role_key`) unique index — and gains `variant_id`, `state` (the full machine, not just submitted/blocked), `intent_id`, `evidence_path`, `outcome`, `campaign_id`, `kit_version`. The old `cc_resume_stats` had to *estimate* resume usage from role titles because rows never recorded the variant; v2.1 records it, so the usage ranking is exact.
- `token_usage` keeps `run_id` as primary key with a date index (the old pattern), and gains a per-stage breakdown — every run's coordinator measures real spend from the platform metering (`agent.agent_message_token_usage` over its agent subtree) and records it via `token_record` with `usage_reported`; unmeasurable runs are flagged unreported, never zero-filled (v2.8).
- `runs` extends the old (timestamp, goal, status, note) with counts, tokens, needs_me, kit_version.
- Dropped: `linkedin_activity`, `content_items`, and the presence/content columns of `daily_metrics` — out of scope. The append-only activity pattern survives in the replies ledger.

13.3 Design system (carried over verbatim)

Graphite/mint/cyan palette, IBM Plex Sans, cards, ledger rows, badges, phone-first responsive with safe-area handling. KPI-first on every tab: summary band (hero number + metric grid), then the ledger. Full numbers with thousands separators — never K/M. **Live data, hard requirement:** every tab reads live on load, with a refresh button that refetches; no static snapshots anywhere. **Honest degradation:** per-section "source unavailable" copy, never sample data; error screens with retry.

KPI band (Applications tab, exact): Today Applied · Chicago time (hero), total applications, submitted, blocked, rejected, parked, submitted · 7 days, submitted · 30 days.

13.4 Ask — deterministic, no LLM

The hub proved the pattern: keyword intent routing (`applications`, `resumes`, `replies`, `runs`, `tokens`, `blockers`, `schedules`, `health`, `unknown`) with hand-written aggregation per intent and per-intent "I can't answer that — the X ledger is unavailable" fallbacks. No model inference, fully deterministic, every answer cites its source. `snapshot("ask")` keeps this contract. Schedule answers now read `schedules_manifest.json` instead of frontmatter scans.

13.5 Build rule

Never declare a tab live before `snapshot()` returns real rows for it. The doctor pings every view hourly and fails loudly. The old fleet retires only after all seven tabs are green for 7 days.

13.7 Evidence opens from the row that recorded it

Every submitted row shows which resume was uploaded (filename + full sha256), whether the confirmation screenshot was captured, and the confirmation text. The links call `file_open` with the row's own `app_id` — record-bound, so a row can only ever open the file attached to itself; arbitrary paths are refused. A lost screenshot renders as "not captured", never a dead link. Resume Library rows open the registered variant PDF the same way (`file_open({variant_id})`). Both the library link and each application's "resume used" link open an in-page PDF viewer modal with Close and Download controls; devices that cannot render embedded PDFs get the Download fallback instead.

13.8 Profile writes — `profile_save`, not `profile_put`, from the UI

Reads go through `snapshot(view)`. Profile-form writes go only through `profile_save`, used by the Profile tab and the first-run wizard; schedule and resume controls use their purpose-built actions in sections 13.10 and 13.11. `profile_save` validates (required keys, types, enums — including `role_types` with at least one of the five allowed values and the required `targeting_titles` list: min 1, non-empty strings), rejects placeholder markers, renders and re-validates the YAML, writes `~/workspace/profile.yaml` through the allowlisted `writeProfileYaml` privileged contract, then `profile_put` — succeeding only when both land (section 7.3). `titles` and `role_types` persist to both YAML and the database and round-trip through `profile_get`; either edit is a profile change, so `profile_watch` auto-reconciles it within ~15 minutes (section 7.4). Direct `profile_put` calls remain the province of `compile-schedules` (YAML → DB sync at compile time); the dashboard never calls it directly. This keeps exactly one writer per direction: screen → validated action, file → `compile-schedules`.

13.9 The Replies tab is the activity ledger (v2.7, date-range filter in v2.9)

The old `replies-ledger` page died for a topological reason, not a product one: five artifacts meant five databases, and the page could never call `harness-core` actions from a sibling runtime. The ledger survives as the Replies tab — one table, newest first, email and LinkedIn rows interleaved: Date (Chicago) · Channel pill (from the thread id) · Person/Topic (leading descriptor of the reason) · Action pill (Sent/Held/Skipped/Auto-sent) · **Rule pill (the row's exact `ruleId`)** · Reason/Source (full text, expandable). No schema change: `reply_log` rows already carry `ruleId` on write, and the tab reads it from `snapshot("replies")`. The tab keeps its KPI header (sent/held counts, funnel, awaiting-me) above the ledger. **v2.9 adds a date-range control** (Today / Last 7 days / Last 30 days / All presets plus a custom start/end picker, inclusive Chicago calendar days) that composes with the channel, action, and search filters — a row shows only when it passes all of them; default All.

13.10 The Schedules tab — per-schedule drift tracking restored (v2.10)

The old fleet's `scheduled-runs` page tracked configuration drift per schedule; the consolidation dropped the page and kept only the per-run drift pill on the Runs tab. The Schedules tab restores it: one row per manifest job, read through `schedules_status` — job name, campaign, cadence, enabled state, last run (America/Chicago), manifest body hash (truncated, full hash in tooltip), and a drift pill reusing the Status tone convention (`drift` → warn tone). Drift derivation: the campaign's latest run row's own `drift` value first (the compiled-vs-live comparison the run-coordinator already performs), direct manifest-vs-live hash comparison when both hashes are present, `unknown` when the campaign has no runs yet. Manifest missing → honest empty state ("No schedules manifest found — run compile-schedules"); action failure → error state with retry; never invented rows.

In v2.13 every one of the 12 rows also has an enable/disable switch and editable cadence. Changes call `schedule_update(job_id, cadence?, enabled?)`, which validates the job id and supported cadence grammar, then updates only that job's `campaigns` entry in `profile.yaml`. Disabling writes `enabled: false`; a job is never deleted. Unknown job ids and unsupported cadence values fail loudly, leaving the file unchanged. The normal `profile_watch` path then validates, recompiles only changed bodies, verifies the live cron records, and rewrites the manifest.

13.11 The Resumes tab — upload, recoverable delete, and viewer (v2.13)

Upload PDF accepts only a file whose bytes begin with `%PDF-`; the extension alone is not trusted. `resume_upload` sanitizes the supplied name, selects a unique filename under `workspace/user/files` without ever overwriting an existing file, computes the full SHA-256, and opens a metadata form. `variant_id` is suggested from the filename but must be unique; a duplicate fails loudly (unlike the administrative `resume_register` upsert). `role_family` is required; `industry_tags` is optional; `years_matrix` starts from the profile when present, otherwise empty; `keyword_vector` starts empty.

Each row has **Delete** with a "Used in N applications" confirmation based on exact references. On confirmation, `resume_delete` deregisters only the `resume_variants` row and moves the source PDF to `~/workspace/.trash/resumes/<variant_id>__<filename>`. Nothing is permanently deleted. Application history and tailored PDFs under `goals/<campaign>/hidden_files/` are never touched. Historical application rows remain readable: `file_open` falls back to the trash location when a deregistered source-library file is still referenced. No table is added and no schema changes.

13.12 First-run setup replaces the tabs until a valid profile exists (v2.13)

When `profile_get` returns null, Career Harness renders no dashboard tabs. It renders a six-step wizard — identity, work auth, employment types, targeting/titles, screening answers, caps — using the same section components, validation rules, and error presentation as the Profile tab. It saves through `profile_save`: validated YAML first, then the database. Placeholder values such as `[FILL IN]` are rejected on both client and server. After a successful save, the completion screen explains that `profile_watch` will compile the 12 schedules automatically within about 15 minutes; the seven normal tabs then become available.

14. Coordinator and agent orchestration

- **The coordinator is the run-coordinator skill**, invoked by each campaign's thin cron body. It opens the run, resumes in-flight rows, pipelines the skill chain, reconciles every ~5 minutes from `snapshot(run)`, and closes the run.
- **Tree depth 2, enforced:** main/worker → skill subagents → (browser tasks). A worker that needs help goes back to the coordinator via the store; it doesn't spawn deeper.
- **Claim, don't race:** parallel workers never own one row — judges return verdicts, the coordinator transitions.
- **Quiet ≠ stuck:** a browser task looks silent for 20 minutes. Handoffs can lag 30+ minutes — the coordinator waits for expected handoffs under 90 minutes old; older ones belong to the doctor.
- **Irreversible actions:** check the store before repeating. A missing report doesn't mean nothing happened — the intent log says what happened.

15. Run lifecycle: trigger to chat in seven steps

1. **Saved.** compile-schedules renders the thin body; `cron.update` stores it; the manifest hashes it.
2. **Fired.** Runtime mints `run_id`, wakes the worker with the body.
3. **Executed.** The worker invokes the run-coordinator skill: `run_open`, resume, spawn skill workers, `browser.spawn_task` per application.
4. **Reported back.** Handoffs arrive asynchronously; the coordinator reconciles, never polls in a loop.
5. **Bookkept.** `run_close` (refuses with unresolved `applying` rows), manifest already current, `snapshot` reflects the run.
6. **Handed to main.** Delivery rule: surface what's new or requested; suppress `nothing_to_do`.
7. **Delivered.** One short chat message — submitted, parked/blocked with reasons, needs-me decisions. Counts from `run_close`, never invented.

16. Reliability

Failure runbook

Failure	Harness behavior	Operator action
Browser handoff lag (30+ min)	Intent log + verify-only task after 90 min; never re-submit blind	None unless unresolved after 2 runs
CAPTCHA / bot wall	<code>parked</code> + checkpoint; retry once next run; 3 parks → <code>companies.skip</code> , operator enqueued	Add to skip list if recurring
SMS verification	<code>parked</code> + queue item for in-window entry	Customer completes manually
Email OTP	Portal-navigator asks coordinator; coordinator fetches via connector; replies via <code>subagent.send</code> ; ≤10 min else <code>parked</code>	None
Unknown mandatory question	screening-answerer → <code>approval_enqueue</code> ; answer lands in profile answers — never asked twice	Answer once
Wrong file attached	Hash re-verification before submit; mismatch → <code>blocked</code>	None — prevented by construction
"Already applied" banner	<code>blocked</code> reason <code>duplicate_portal</code> + screenshot	None — scout learns
Worker dies mid-stage	Row keeps state; next coordinator resumes	None
Coordinator dies	Next run resumes from recorded state	None
<code>harness-core</code> unreachable	Workers append to <code>spillover.jsonl</code> ; doctor replays via <code>spillover_replay</code>	Investigate artifact
Body drift (hash ≠ manifest)	Doctor reports; <code>profile_watch</code> auto-recompiles within ~15 min	Manual compile-schedules remains as fallback; never hand-edit
LinkedIn restriction warning	All LinkedIn campaigns auto-pause	Incident call within 24h
<code>hidden_files</code> > 2 GB	Doctor reports oldest run folders to prune	Prune (never auto-delete)

harness-doctor (hourly, the skill)

Reads only `harness-core` + `cron.view`. Checks: body-hash drift · `applying` > 90 min → verify-only task · `approvals` > 24h · runs without `run_close` > 3h → mark `failed` · company parks ≥ 3 → skip + enqueue · H-1B data age > 120d · disk > 2 GB · `snapshot()` liveness. Output: `{"status": "green|amber|red", ...}`. **Green** → **silent**. Amber/red → one message with the fired rows. Hard limits: never re-submits, never edits bodies, never deletes files. Note: the doctor template's line telling the operator to run compile-schedules manually is now stale — `profile_watch` (section 7.4) handles recompiles automatically; manual recompile stays available as a fallback.

Token budget

Measured per-run tokens, not estimates (v2.8): before `run_close`, the coordinator sums the platform's own per-message metering over its whole agent subtree and records it with `token_record` (per-stage breakdown in `runs.tokens`, totals in `tokens_input / tokens_output / tokens_total`). Runs that can't be measured are flagged `tokens_reported=false` and display as "—" — never 0, never backfilled. Screen-before-claim keeps the expensive stages (tailor, review) to rows that can actually submit. **Target: ≤ 60% of v1 tokens per submitted application.** Per-goal budgets, 90% soft-stop, weekly burn review.

17. Filesystem and visibility contract

```
~/workspace/
├── skills/                # THE logic layer: 18 versioned skills + READMEs
├── profile.yaml           # the single rule source (human-editable)
├── templates/            # campaign body templates + profile.schema.yaml
├── schedules_manifest.json # compiled-vs-live truth
├── goals/<campaign>/      # thin: GOAL.md + compiled body + files/ + hidden_files/
├── user/files/           # resume variant sources
├── your_files/           # the ONLY inbox for customer deliverables
└── migration_archive/    # retired files – moved, never deleted
```

Location	Who sees it	What's in it
goals/<g>/files/	Customer, Goals tab	Documents they asked for
goals/<g>/hidden_files/	Agent + workers	Blobs and scratch — evidence, never state
goals/<g>/hidden_files/<run_id>/screenshots/	Agent + workers	Submission evidence: <app_id>_<step>.png screenshots, <app_id>_confirmation.png, <app_id>_confirmation.txt. Canonical paths; the dashboard opens them only through <code>file_open</code> (section 13.7)
your_files/	Customer, in chat	Deliverables handed over
skills/	Agent + workers (+ ships in the kit)	All logic, zero personal data
harness-core	Dashboards, workers (actions only)	All state, all truth
~/memory/	Main agent only	Narrative continuity

18. Memory and judgment — the hard boundary

Hot (`memory/YYYY-MM-DD.md`) / Warm (`MEMORY.md`) / Cold (`bank/`) / Derived (alignment synthesis) — same as v1. But v2.1 draws the line v1 blurred: **memory is narrative; it is never a rule source.** Scheduled workers don't get memory — they get a compiled body and `profile_get`. Customer corrections become `profile.yaml` edits, `companies.skip` flags, and skill improvements via the learning loop — not notes in a log.

19. Install-ready: the kit as software

"Runs on each system" is engineered, not hoped for.

- **Semver kit versions** (1.0.0, 1.1.0...). Every run, event, and manifest row is stamped with the kit version that produced it. Skill frontmatter carries per-skill versions.
- **Idempotent installer** (a runbook + the compile-schedules/doctor skills, runnable end-to-end by the operator):
 1. Scaffold goal directories from skeletons.
 2. Install the skill catalog to `~/workspace/skills/` (unchanged).
 3. Build the `harness-core` artifact; run Drizzle migrations in order.
 4. Seed: `h1b_import`, `companies_import`, resume tagging.
 5. Launch Career Harness. A fresh install has no profile row, so the six-step setup wizard collects and validates the profile, writes `profile.yaml`, then calls `profile_put`; `[FILL IN]` markers are never installed as data.
 6. Let `profile_watch` compile schedules automatically within about 15 minutes (manual compile-schedules remains an operator fallback), then verify the manifest with `cron.view`.
 7. **Smoke tests:** doctor green + a dry-run scout (scout skill with `gates: review_all`, zero submits) producing ≥ 5 `discovered` rows.
 8. Declare live only when all seven pass. Re-running the installer is always safe.
- **Migrations, not rebuilds.** Schema changes ship as Drizzle SQL migrations applied in order; the upgrade playbook is: backup manifest → apply migrations → recompile → doctor green. No customer data is ever re-entered.
- **Kit contents (ships):** 18 skills + READMEs · 13 templates (12 body templates + `profile.schema.yaml`) · empty goal skeletons · `harness-core` source (client + server + migrations) · seed CSVs · blank tables · installer runbook · this blueprint. (v2.12: `profile.schema.yaml`, the `job-board-search` and `linkedin-feed-hunting` skills, and `profile.example.yaml` updated for `targeting_titles`; template count unchanged at 13.)
- **Never ships:** resumes, applications, memory, people pages, credentials, connected accounts, token history, any customer's corrections or outcomes.

20. Why this beats the market

The competition (one-click apply SaaS, browser extensions, spreadsheets-with-ambition) automates *clicking*. We install *judgment* — and prove every action. Six moats, each structural:

1. **Calibration as a service.** Shadow week + correction-rate metric: the system demonstrably converges on the customer's judgment. SaaS tools have settings; we have a convergence curve.
2. **Proof of every action.** Intent log, hash-verified uploads, per-page screenshots, confirmation strings. When something goes wrong, the evidence is already on disk. Nobody else in this market can show their work.
3. **Outcome-tracked learning.** The loop optimizes reply rate, not submission count — sources, variants, and tiers ranked by interviews, not volume. Competitors celebrate "500 applications sent."
4. **The H-1B data moat.** USCIS + LCA data compiled into a lookup turns visa sponsorship from a gamble into a scored filter. For H-1B candidates, this alone beats every generic tool.
5. **Privacy as architecture.** Everything runs in the customer's own environment; data leaves only by their explicit export. SaaS competitors hold your credentials, your resume, and your job search on *their* servers.

6. Compounding skill IP. Every portal quirk fixed, every playbook refined, every judge threshold tuned ships in the next kit version to every customer — while no customer's personal data ever moves. The product gets smarter; the data stays home.

21. Reconciliation with N-Pulse (what carried over, what died)

The v1 N-Pulse deployment was the laboratory. Explicitly:

N-Pulse element	Verdict in v2.1
Files-as-blackboard, workers-as-mayflies	Carried over — now split: state → DB, blobs → files, logic → skills
15 hand-written cron bodies	Died — replaced by compile-schedules + thin bodies
<code>applications.csv</code> + merge rule (the 37-vs-25 wart)	Died — single store, constraints enforce
Phased pipeline proposal (Scout → Tailor → Apply)	Carried over — now the run-coordinator skill, pipelined
resume-reviewer gate, approval-judge, honesty rules	Carried over unchanged — now versioned skills
"Wait means stop", silence-by-default delivery	Carried over unchanged
Per-goal <code>hidden_files/</code> run folders	Carried over — now evidence-only, never scanned for state
Memory as coordination	Died — memory is narrative; the store coordinates
Presence/Content modules in the same design	Excluded — separate add-ons, never in the core kit
Addepar hash-verification lesson	Carried over — now enforced inside portal-navigator

Nothing from N-Pulse was discarded without a named replacement. That's the point of this section: the next operator inherits the *reasoning*, not just the artifacts.

22. Build order

Phase	Deliverable	Exit test
0	Skill catalog freeze: 18 skills, contract + conventions + READMEs reviewed; <code>profile.schema.yaml</code>	Every skill passes a dry-run invocation returning valid verdict JSON
1	<code>harness-core</code> : <code>schema</code> , <code>app_claim</code> , <code>app_transition</code> , <code>profile_put</code> , <code>review_get/put</code> , <code>snapshot</code> , <code>spillover_replay</code>	Nightly run writes only through actions; dual-write proves DB, then CSVs retire
2	run-coordinator skill driving the phased workflow; intent log + hash check in portal-navigator; inbox skills on the ledger	One full night: zero double-applies, every <code>applying</code> row resolved, replies logged with rule ids
3	compile-schedules + templates + manifest + drift check	All bodies regenerated from source; hashes match <code>cron.view</code> ; a chat-stated rule is provably not live until compiled
4	Judge skills + <code>resume_pick</code> ; H-1B import + h1b-judge	Every claimed row carries judge reasons + H-1B score in <code>events</code>
5	Portal playbooks (6 ATS + generic) + <code>career_portal</code> campaign	≥ 80% of tier-1 portals reach confirmation without a park
6	<code>harness-doctor</code> + verify-only tasks; seven Career Harness tabs on <code>snapshot()</code> plus the focused write actions in §13 (one artifact, exact variant-level resume stats)	Doctor green 7 days; old fleet retired
7	Installer runbook + packaging script; hooks pilot for inbox alongside hourly scan	Customer #1 live in one session + shadow week; installer re-run is a no-op

23. Productizing: packaging and onboarding

Onboarding (one working session + shadow week): environment → install kit → Career Harness six-step setup → resume upload/tagging → seed data → automatic profile-watch compile → **smoke tests** → shadow week (everything `draft_for_review`, cap 3/run, ≥ 20 reviewed rows) → go live (first `submitted` with confirmation screenshot) → weekly tune from the customer's exported `snapshot` JSON (changelog in their `files/`).

What the operator keeps: templates, skills, seed datasets, artifact source, installer runbook, onboarding playbook. **What stays with the customer:** everything in their environment. Pricing can key off `runs` and `applications` counts from `snapshot` — shareable without exposing anything else.

24. Learning loop

Signal	Where it lands	What it tunes
Customer rejects a tailored resume	<code>reviews</code> (verdict + notes)	<code>resume_variants.approval_rate</code> → picker weight; notes as variant gaps
Customer answers an unknown screening question	profile answers	Never asked again; compiled into the next body
"Never this company / role"	<code>companies.skip</code> + reason	Scouts never fetch; <code>app_claim</code> refuses
Judge rejected, customer overrides	<code>events</code> (override)	Threshold nudged; 3 overrides on one rule → flagged for operator
Portal parks 3×	<code>companies</code> park count + ATS type	Playbook gap list; deprioritized until fixed
Submission confirmed by email	<code>applications.confirmed</code>	Verify-stage precision
Interview or reply received	<code>applications.outcome</code>	Sources, variants, tiers ranked by reply rate
Skill misfires (bad verdict, bad reply)	skill version flagged in <code>events</code>	Next kit version fixes the skill for <i>every</i> customer

The last row is new in v2.1 and it's the compounding moat: a skill fix ships once and helps everyone, without any customer's data moving.

25. Open decisions — what I recommend

#	Decision	Recommendation	Why
D1	Pilot phased pipeline on nightly or separate schedule?	Nightly; 7am as control for 2 weeks	Real traffic, real comparison; a separate schedule splits cap math
D2	Concurrent appliers: 2 or 3?	2 in pilot, 3 after a clean week	LinkedIn account risk dominates throughput
D3	Keep the 10-per-run cap?	Keep as kit default; per-customer at intake	Pacing control, not quality control — judges do the filtering
D4	<code>h1b_gate</code> default for new customers	Soft (hard available at intake)	Hard rejects below 40 silently kill pipelines; customers tighten after seeing data
D5	Inbox on the ledger: Phase 2 or 6?	Phase 2	Outcome tracking is the only real metric; don't optimize submissions for five phases
D6	Where does the operator read <code>snapshot</code> for tuning?	Customer-exported JSON	Customer controls what leaves; a shared card is a standing pipe — wrong default
D7	CSV cutover: hard or dual-write?	Dual-write in Phase 1 only, DB authoritative, then retire	Flag-day cutovers on real job applications are reckless
D8	<code>resume_pick</code> weights — hand-tune early?	No. Ship 0.45/0.25/0.20/0.10, retune monthly	Early tuning overfits to ten data points
D9	Telemetry for Managed tier	Opt-in required	Can't operate blind; must be explicit
D10	Max agent tree depth	2, enforced	Deeper trees orphaned work in v1
D11	Hooks for inbox: pilot now or later?	Pilot in Phase 1, alongside hourly scan	Push cuts triage latency ~60min → ~2min; the watermark scan is the backstop if hooks misfire
D12	Skill versioning policy	Semver kit + per-skill frontmatter; manifest records both	"Why did Tuesday behave differently?" must be answerable from the manifest

26. Risks

Risk	Level	Mitigation
Platform terms (LinkedIn automation)	High	Rate caps, human cadence, drafts for creative work, auto-pause on first warning, customer-owned accounts, explicit consent
Fabricated or inflated claims	High	Honesty rules in every body; reviewer gate; hash-verified uploads; profile as the only answer source
<code>harness-core</code> single point of failure	Medium	Doctor pings <code>snapshot()</code> hourly; spillover + idempotent replay; versioned, rebuildable artifact source
Skill drift across kit versions	Medium	D12: manifest records skill versions; doctor never auto-recompiles; upgrades are explicit
Muse primitive changes	Medium	Logic in portable skills + standard SQL; no platform tricks where a file/row convention works
Client PII	Medium	Never leaves the environment except by explicit customer export
Token cost	Medium	Per-goal budgets, 90% soft-stop, screen-before-claim, per-skill spend in every verdict
Non-convergence	Medium	Correction-rate tracking; extended shadow mode; kill criteria at intake

27. Roadmap

- **Phase 0 · Oct** — Prove it. Skill catalog freeze; pilot the phased skill chain on the reference deployment; ship the schema + intent log + single-store cutover. Measure wall-clock and pass rates.
- **Phase 1 · Nov** — Package it. Kit 1.1: skills, templates, schema, rule compiler, installer runbook, smoke tests. Dashboard spec reconciled against the old dashboard code.
- **Phase 2 · Dec-Jan** — Five clients. Launch tier with five design partners. Tune onboarding to calibration inside three weeks. Hooks pilot for inbox.
- **Phase 3 · Q1** — Operate at scale. Kit 2.0: opt-in telemetry, ops console, Managed tier, second operator. Evaluate deterministic-workflow coordinator.

The principle underneath all of it

We don't automate a job search. We install a person's judgment into a system that never sleeps — and prove every action it takes.